

GESEC DESAFIO DEV PYTHON



Contexto do Desafio

Sistemas de Gerenciamento de Vídeo (VMS) modernos exigem alta eficiência no processamento de múltiplas transmissões simultâneas. O grande desafio é garantir que a captura, o decodificar do vídeo (via hardware) sem impactar a fluidez da interface gráfica (UI).

Seu objetivo é construir uma aplicação simplificada de visualização de câmera que demonstre sua capacidade de lidar com streaming, **multithreading** e **renderização eficiente**. Crie uma interface gráfica com python que reproduza pelo menos dois quadrantes de vídeos.

1. Interface Gráfica (PyQt5)

- Uma janela principal com uma área de exibição de vídeo (`QLabel` ou `QOpenGLWidget`).
- Botões de **Iniciar**, **Pausar** e **Parar** o streaming.
- Um contador de FPS real da renderização exibido na tela.

2. Gerenciamento de Threads e Concorrência

- **Proibido travar a UI:** A captura e a decodificação dos frames *devem* rodar em uma thread separada (`QThread` ou `Worker` utilizando `QThreadPool/signals`).
- Utilização correta de sinais (`pyqtSignal`) para enviar os frames processados da thread de captura para a thread principal da UI.

4. Utilização de GPU (Diferencial Crítico)

- Demonstrar o entendimento de processamento em placa de vídeo.
- *Opção A (Aceleração na decodificação):* Configurar o OpenCV/FFmpeg para ler o stream utilizando os drivers da GPU (ex: `cv2.cuda` se disponível, ou decodificação via hardware via propriedades do FFmpeg).
- *Opção B (Aceleração no processamento):* Aplicar um filtro simples (ex: conversão de cor, redimensionamento ou thresholding) utilizando estruturas que rodem na GPU (como tensores do PyTorch/CuPy ou módulos CUDA do OpenCV).
- *Nota:* Caso o candidato não possua GPU dedicada no ambiente de desenvolvimento, ele deve estruturar o código deixando a lógica/funções prontas para alternar entre CPU e GPU (ex: `device = 'cuda' if available else 'cpu'`).



Critérios de Avaliação

- **Arquitetura e Concorrência (Peso Alto):** Como você separou a lógica de backend (captura) da UI. Uso correto de Mutexes/Locks se houver compartilhamento de memória.
- **Performance:** Consumo de CPU/Memória controlado e estabilidade do FPS na tela.
- **Qualidade de Código:** Organização de classes, clean code, tratamento de exceções (ex: perda abrupta de conexão com a câmera).
- **Domínio de Hardware:** Como o código gerencia a transferência de dados entre a memória do sistema (Host) e a memória da placa de vídeo (Device).



Entregáveis

1. **Código-Fonte:** Link para um repositório Git (GitHub / GitLab).
2. **README.md:** Instruções claras de como rodar o projeto, dependências do sistema (drivers CUDA, FFmpeg, etc.) e uma breve explicação da arquitetura que você escolheu para o gerenciamento das threads.

Observações:

1. Recomendamos utilização de ferramentas gratuitas caso contrário os custos das mesmas serão de total responsabilidade do candidato.
2. O candidato que não entregar o teste na data solicitada na entrevista será desclassificado do processo seletivo, em caso de imprevistos, comunicar ao recrutador.
3. Os testes entregues na data, mesmo se incompletos, serão validados e avaliados